

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA TOÁN - TIN HỌC

—o0o—



BÁO CÁO ĐỒ ÁN
PYTHON CHO KHOA HỌC DỮ LIỆU

ĐỀ TÀI:
DỰ ĐOÁN KHỐI LƯỢNG CHẤT BÉO BÃO HÒA

Giảng viên : Hà Minh Tuấn

Họ tên sinh viên : Phạm Khang Bình
MSSV : 232380043

Nguyễn Hoàng Anh Duy
23280051

Nguyễn Hải Lâm
23280068

Lớp : 23KDL
Nhóm : 4

TP. Hồ Chí Minh, ngày 10 tháng 12 năm 2025

Mục lục

1	Giới thiệu bài toán và dữ liệu	2
1.1	Mô tả dữ liệu và EDA	2
1.2	Phân tích dữ liệu	6
1.3	Ý nghĩa thực tiễn của bài toán	6
1.3.1	Đối với người nội trợ	7
1.3.2	Đối với người ăn kiêng và người tập luyện	7
1.3.3	Đối với người bình thường có nhu cầu theo dõi sức khỏe	7
1.3.4	Ứng dụng trong tương lai	7
2	Giới thiệu các Class và phương thức trong project	7
2.1	Cấu trúc thư mục dự án	8
2.2	Package: src.data	8
2.2.1	src/data/io.py	8
2.2.2	src/data/transformers/	9
2.2.3	src/data/preprocessor.py	11
2.3	Package: src.models	12
2.3.1	src/models/evaluator.py	12
2.3.2	src/models/io.py	13
2.3.3	src/models/trainer.py	13
2.3.4	src/models/optimizer.py	14
2.4	Package: src.visualization	14
2.4.1	src/visualization/eda.py	15
2.4.2	src/visualization/model_plots.py	15
3	Tổng quan thư viện và phương pháp tối ưu siêu tham số	16
3.1	Thư viện học máy sử dụng	16
3.2	Các mô hình sử dụng và siêu tham số chính	16
3.3	Chỉ số đánh giá mô hình	19
3.4	Tối ưu siêu tham số bằng phương pháp Bayesian	22
3.4.1	Cách hoạt động của Bayesian Optimization	22
3.4.2	Lý do Bayesian Optimization tốt hơn Grid Search và Random Search	22
4	Kết quả thực nghiệm	22
4.1	Chi tiết quá trình tiền xử lý dữ liệu	24
4.1.1	Làm sạch dữ liệu cơ bản	24
4.1.2	Chia dữ liệu Train/Test	24
4.1.3	Xử lý giá trị ngoại lai	24
4.1.4	Xử lý giá trị khuyết	24
4.1.5	Tóm tắt kết quả tiền xử lý	25
4.2	So sánh các mô hình học máy	25
4.3	Phân tích đặc trưng quan trọng của các mô hình	26
5	Bảng công việc	28

1 Giới thiệu bài toán và dữ liệu

Trong project này, bài toán được lựa chọn là **hồi quy** (regression), với mục tiêu dự đoán **khối lượng chất béo bão hòa (saturated fat) của các món ăn nhanh** tại những chuỗi cửa hàng lớn, dựa trên các đặc trưng dinh dưỡng cơ bản, dễ dàng tính toán được. Bộ dữ liệu sử dụng là *Fast_Food_Dataset*, được công bố trên Kaggle¹.

1.1 Mô tả dữ liệu và EDA

Dưới đây là mô tả chi tiết các trường dữ liệu có trong bộ dữ liệu:

Cột	Mô tả
Company	Tên công ty hoặc thương hiệu cung cấp sản phẩm.
Item	Tên món ăn hoặc sản phẩm cụ thể.
Calories	Tổng lượng calo trong một khẩu phần.
Calories from Fat	Lượng calo có nguồn gốc từ chất béo.
Total Fat (g)	Tổng lượng chất béo (gram) trong một khẩu phần.
Saturated Fat (g)	Lượng chất béo bão hòa (gram).
Trans Fat (g)	Lượng chất béo chuyển hóa (gram).
Cholesterol (mg)	Hàm lượng cholesterol (mg).
Sodium (mg)	Hàm lượng natri/muối (mg).
Carbs (g)	Tổng lượng carbohydrate (gram).
Fiber (g)	Hàm lượng chất xơ (gram).
Sugars (g)	Lượng đường (gram).
Protein (g)	Hàm lượng protein (gram).
Weight Watchers Pnts	Điểm số theo hệ thống đánh giá dinh dưỡng Weight Watchers.

Bảng 1: Bảng mô tả dữ liệu *Fast_Food_Dataset*

Các cột được chọn để loại bỏ trước khi tiền xử lý bao gồm:

- **Company** và **Item**: Đây là các đặc trưng chỉ tên món ăn, tên chuỗi thực phẩm, chúng không mang bất cứ ý nghĩa nào trong bài toán.
- **Calories** và **Calories from Fat**: Hai đặc trưng này có mối quan hệ chặt chẽ với nhau. dựa trên công thức sau:

$$\text{Calories from Fat} = \text{Total Fat (g)} \times 9$$

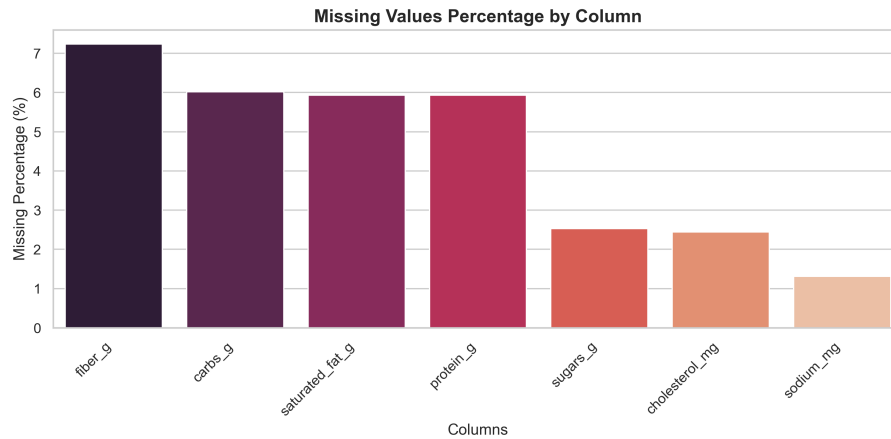
Và ngoài ra Calories có thể được tính như sau:

$$\text{Calories} = \text{Proten (g)} \times 4 + \text{Carbs (g)} \times 4 + \text{Total Fat (g)} \times 9$$

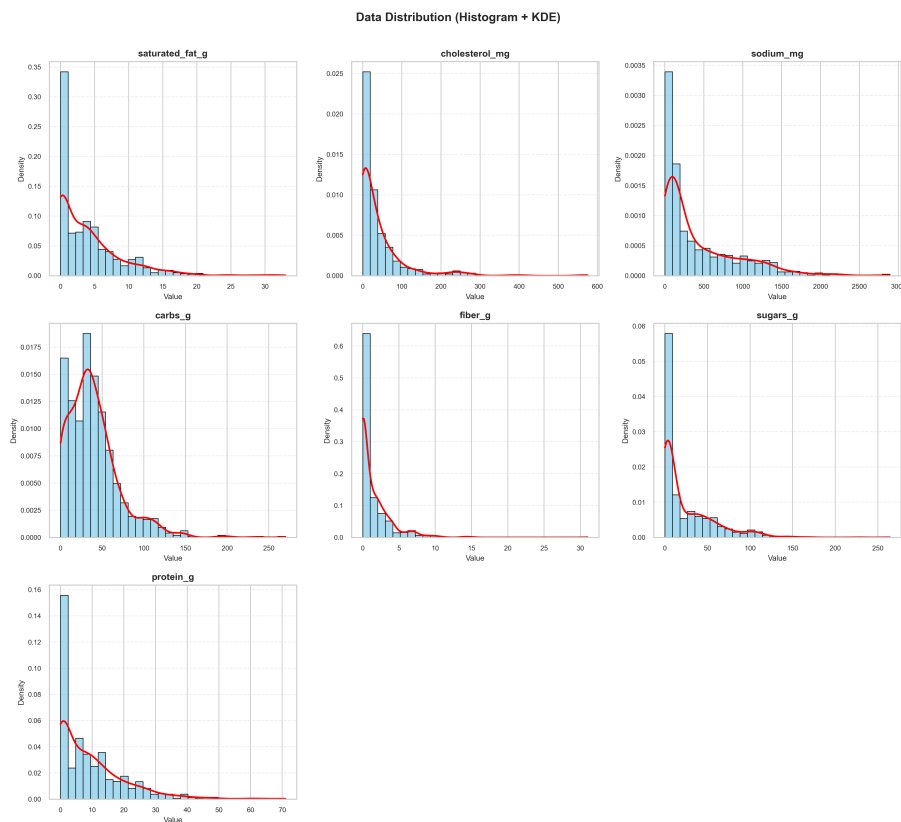
Vậy có thể thấy nếu giữ cả 2 cột này sẽ dễ dàng xảy ra hiện tượng đa cộng tuyến, cho ra kết quả không tốt.

¹<https://www.kaggle.com/datasets/tan5577/nutritonal-fast-food-dataset>

- **Total Fat (g)** và **Trans Fat (g)**: Hai đặc trưng này chỉ có thể đo chính xác bằng thiết bị chuyên dụng². Do đó, trong thực tế người dùng phổ thông khó có thể thu thập được các thông tin này để đưa vào mô hình dự đoán.
- **Weight Watchers Pnts**: Điểm số này được tính toán dựa trên **Calories**, giữ lại không mang nhiều ý nghĩa trong việc dự đoán chất béo bão hòa.

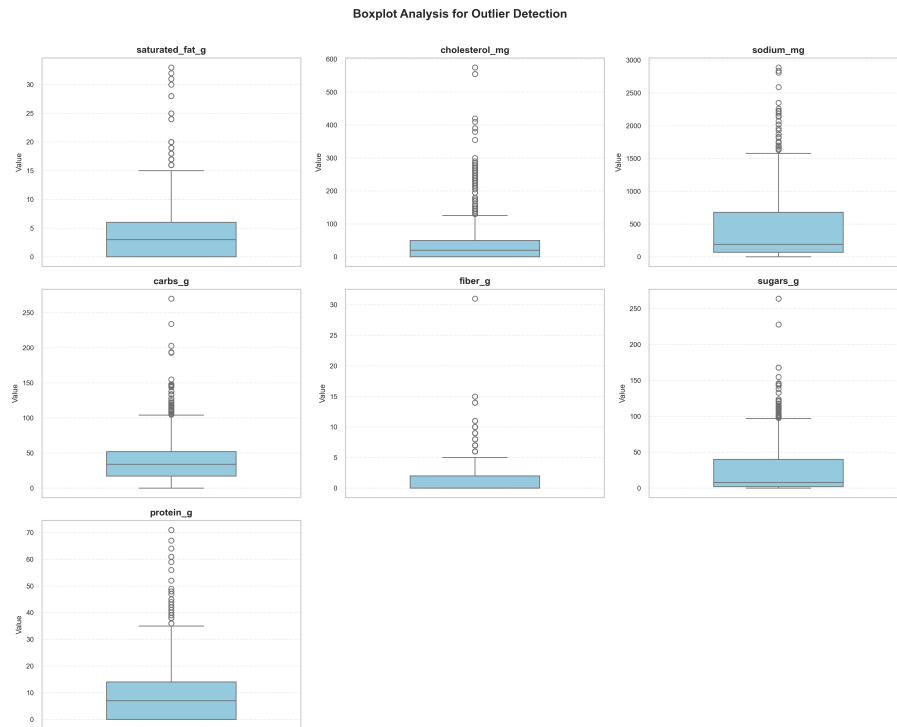


Hình 1: Biểu đồ giá trị khuyết trong dữ liệu

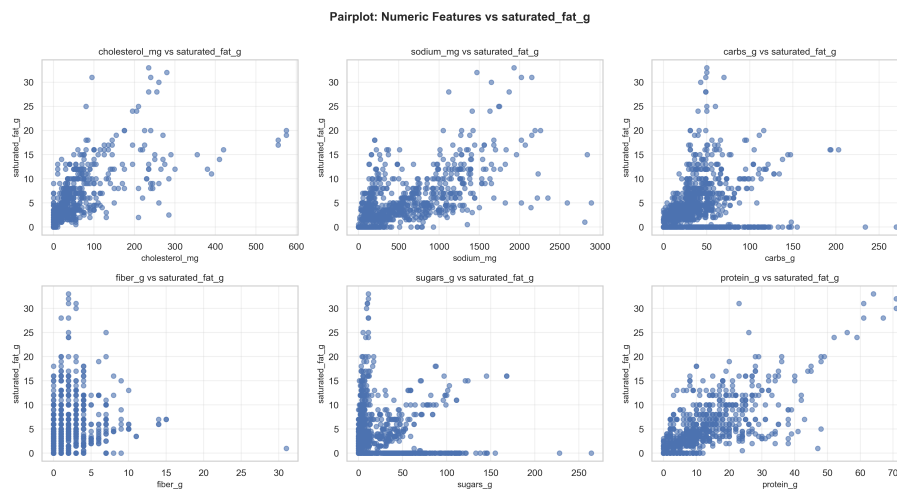


Hình 2: Phân phối dữ liệu các đặc trưng

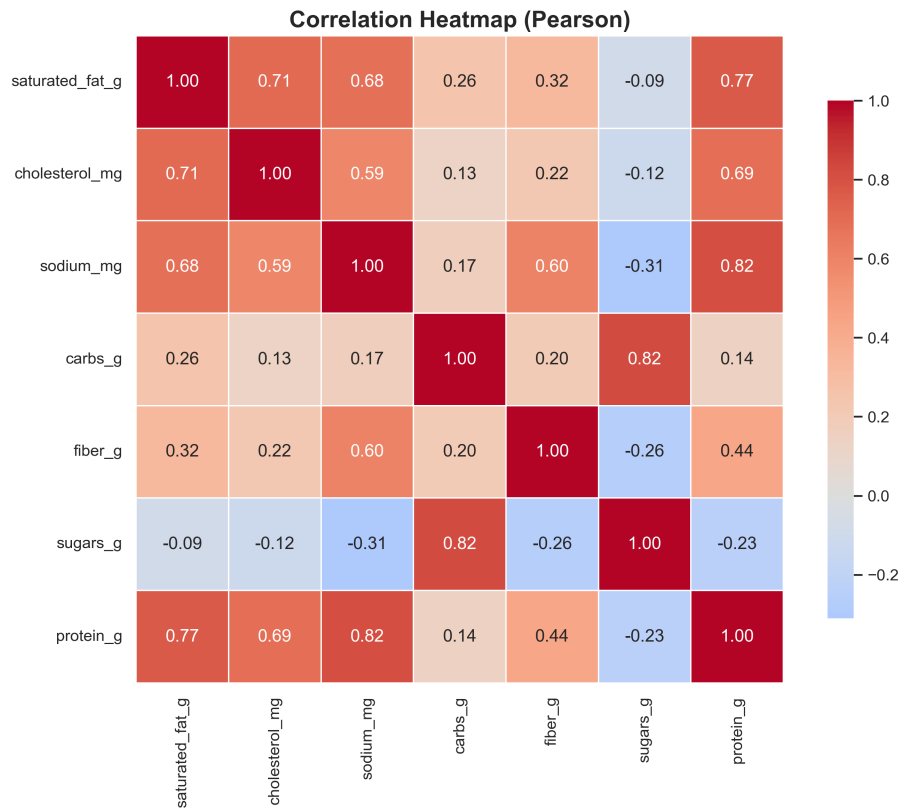
²Khác với Cholesterol và Sodium, ta có thể ước lượng chúng và quy đổi dễ dàng hơn do nó ít bị biến đổi khi chế biến, ngoài ra giá trị chính xác được ghi rất rõ trong các nhãn sản phẩm khi mua hàng



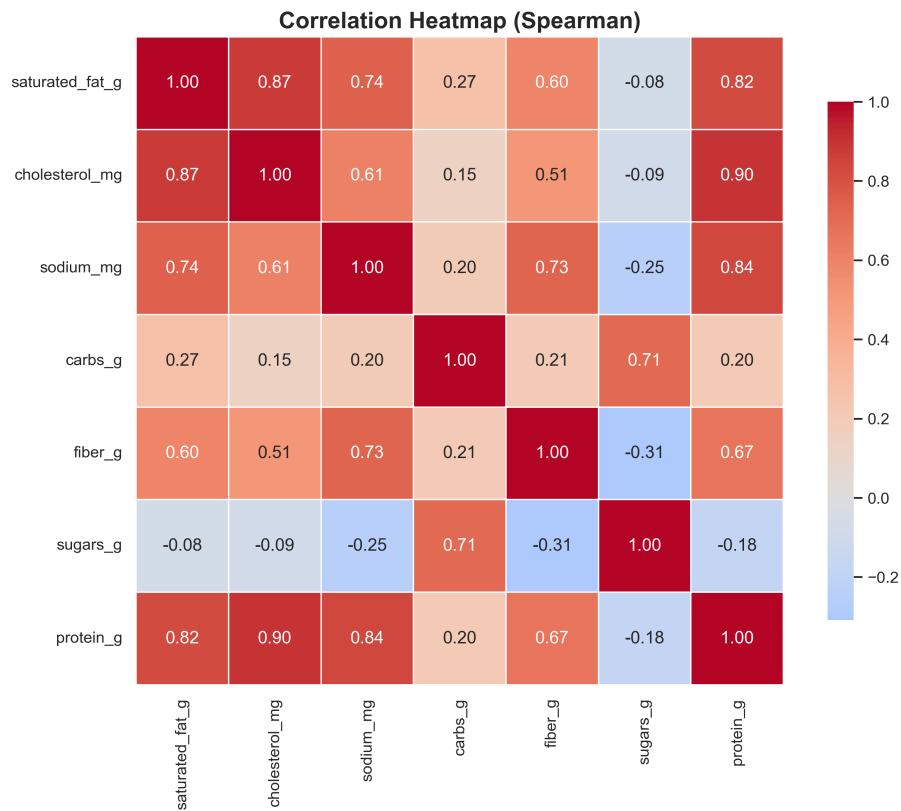
Hình 3: Biểu đồ hộp (boxplot) các đặc trưng



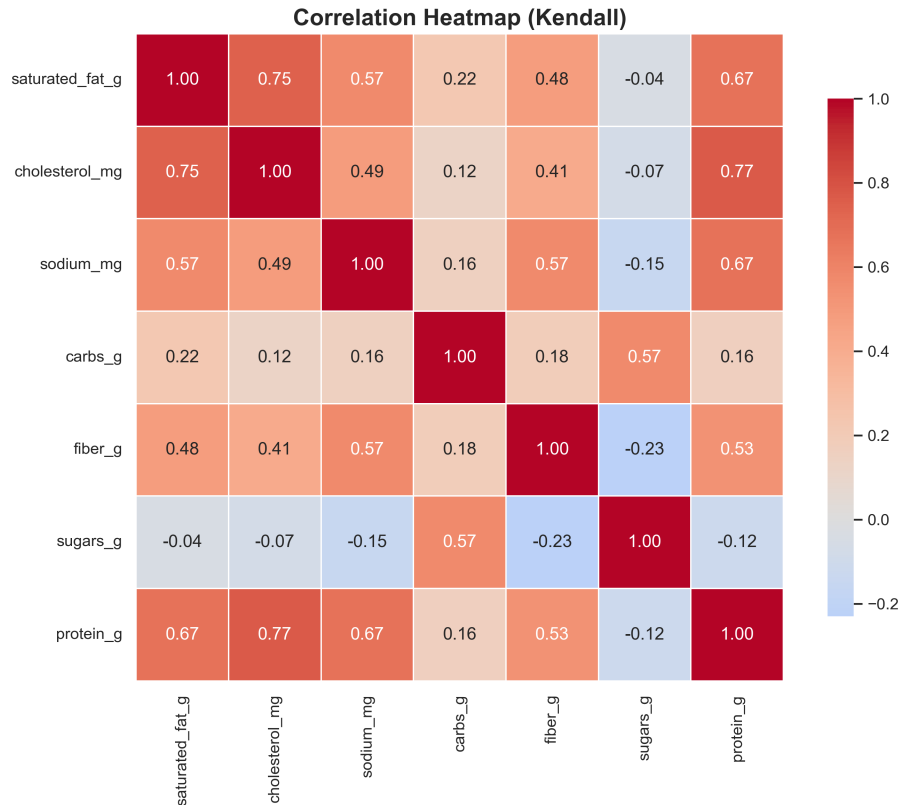
Hình 4: Pairplot các đặc trưng số



Hình 5: Heatmap tương quan các đặc trưng (Pearson)



Hình 6: Heatmap tương quan các đặc trưng (Spearman)



Hình 7: Heatmap tương quan các đặc trưng (Kendall)

1.2 Phân tích dữ liệu

Bộ dữ liệu này có phân phối lệch phải với toàn bộ đặc trưng, chứa khá nhiều giá trị ngoại lai (outliers), ta sẽ phải xem xét để quyết định áp dụng các phương pháp xử lý phù hợp.

Hình 1 minh họa số lượng giá trị khuyết trong từng đặc trưng, điều đáng chú ý là cột mục tiêu **saturated_fat_g** cũng có một lượng giá trị khuyết, ta sẽ phải bỏ các hàng bị khuyết này đi, ngoài ra khi xem xét kỹ hơn, các hàng này đều chứa giá trị khuyết ở hầu hết các đặc trưng của nó đây chính là lỗi nhập liệu.

Dựa trên heatmap tương quan ở Hình 5, Hình 6 và Hình 7, ta rút được một số nhận xét quan trọng sau:

- **cholesterol_mg**, **sodium_mg** và **protein_g** là các đặc trưng có tương quan cao nhất với mục tiêu.
- Đặc trưng **protein_g** có tương quan khá cao với các đặc trưng khác, điều này có thể dẫn tới đa cộng tuyến nếu không cẩn thận, ảnh hưởng tới hiệu năng mô hình.

Dựa trên ngữ nghĩa thực tế, ta sẽ **chuyển toàn bộ giá trị âm thành giá trị dương** (nếu có tại đây chính là lỗi nhập liệu—typo).

1.3 Ý nghĩa thực tiễn của bài toán

Việc ước lượng hàm lượng chất béo xấu (bao gồm *saturated fat* và *trans fat*) từ các chỉ số dinh dưỡng dễ thu thập như *cholesterol*, *sodium* (dựa vào nhãn thành phần), *carbohydrate*,

fiber, *sugars* và *protein* (có thể cân dễ dàng ở nhà) mang lại nhiều ý nghĩa thực tiễn quan trọng trong đời sống hằng ngày, đặc biệt trong bối cảnh người dùng phổ thông không có điều kiện đo lường chính xác các thành phần chất béo bằng thiết bị chuyên dụng.

1.3.1 Đối với người nội trợ

- Ước lượng nhanh mức độ chất béo xấu trong bữa ăn hằng ngày.
- Điều chỉnh lượng gia vị, dầu mỡ và các thực phẩm có nguồn gốc động vật.
- Chủ động bảo vệ sức khỏe tim mạch cho các thành viên trong gia đình.

1.3.2 Đối với người ăn kiêng và người tập luyện

- Duy trì tỷ lệ mỡ cơ thể ở mức hợp lý.
- Hạn chế nguy cơ rối loạn mỡ máu trong quá trình ăn kiêng kéo dài.
- Tối ưu hoá hiệu quả tập luyện thông qua chế độ dinh dưỡng lành mạnh.

Mô hình dự đoán giúp nhóm đối tượng này kiểm soát chất béo một cách chủ động ngay cả khi không có sẵn thông tin chi tiết trên nhãn thực phẩm.

1.3.3 Đối với người bình thường có nhu cầu theo dõi sức khỏe

Đối với người trưởng thành bình thường, việc duy trì lượng *saturated fat* trong giới hạn khuyến nghị (dưới 10% tổng năng lượng mỗi ngày, lý tưởng dưới 7%) giúp giảm đáng kể nguy cơ mắc các bệnh tim mạch, cao huyết áp và tiểu đường type 2. Hệ thống ước lượng từ mô hình học máy có thể:

- Cảnh báo sớm khi lượng chất béo xấu vượt ngưỡng an toàn.
- Hỗ trợ người dùng điều chỉnh khẩu phần ăn hợp lý theo từng ngày.
- Hình thành thói quen ăn uống khoa học và bền vững.

1.3.4 Ứng dụng trong tương lai

- Thực hiện triển khai trên các ứng dụng điện thoại.
- Hỗ trợ tích hợp tính toán Cholesterol và Sodium nhanh hơn thay vì phải tìm kiếm nhập liệu thủ công.

2 Giới thiệu các Class và phương thức trong project

Phần này mô tả cấu trúc mã nguồn của project và giải thích **chức năng của từng Class cùng các phương thức (methods) chính**. Hệ thống được tổ chức trong thư mục `src/` và chia thành ba package chính: `src.data`, `src.models` và `src.visualization`. Cách trình bày được thiết kế theo phong cách tài liệu kỹ thuật, giúp người đọc dễ dàng tra cứu khi muốn xem lại luồng xử lý của chương trình.

2.1 Cấu trúc thư mục dự án

```
1 Python-Cho-KHDL/
2 |-- src/                # Source code
3 |   |-- data/           # Xu ly du lieu
4 |   |-- models/        # Mo hinh hoc may
5 |   |-- utils/         # Tien ich
6 |   +-- visualization/  # Truc quan hoa
7 |-- scripts/           # Scripts chay chuong trinh
8 |   |-- train.py       # Script huan luyen chinh
9 |   +-- experiment.py  # Script thuc nghiem
10 |-- configs/          # Cau hinh
11 |   +-- default_config.ini # File cau hinh mac dinh
12 |-- data/             # Du lieu (raw/interim/processed)
13 |-- outputs/          # Ket qua dau ra
14 |   |-- logs/         # Logs huan luyen
15 |   |-- models/       # Models da luu (.joblib)
16 |   |-- plots/        # Bieu do
17 |   |   +-- eda/      # Bieu do EDA (before/after)
18 |   +-- results/     # Ket qua danh gia (.csv, .json)
19 |-- notebooks/        # Jupyter notebooks
20 |   |-- eda.ipynb     # Notebook EDA
21 |   +-- model_visualize.ipynb # Notebook visualization
22 |-- docs/             # Tai lieu/bao cao
23 |-- requirements.txt
24 +-- readme.md
```

2.2 Package: src.data

Package này chịu trách nhiệm xử lý dữ liệu đầu vào, bao gồm: đọc/ghi file, làm sạch dữ liệu thô, xử lý giá trị khuyết, ngoại lai và chuẩn hoá dữ liệu để đưa vào mô hình học máy.

2.2.1 src/data/io.py

Class: DataIO

Class chuyên trách việc Đọc/Ghi file và chuẩn hóa tên cột. Đảm bảo dữ liệu được nạp vào và lưu ra đúng định dạng, nhất quán với toàn bộ pipeline.

Methods:

`load_data(filepath)`

Nạp dữ liệu từ các file (CSV, XLSX, JSON) vào `pandas DataFrame`. Hỗ trợ xử lý lỗi khi file không tồn tại hoặc sai định dạng, giúp chương trình không bị crash đột ngột.

`save_data(data, filepath)`

Lưu `DataFrame` hiện tại vào file CSV tại đường dẫn chỉ định. Được dùng sau khi dữ liệu đã qua xử lý để lưu lại phiên bản “sạch”.

`clean_column_names(data)`

Chuẩn hóa tên cột: loại bỏ ký tự đặc biệt, khoảng trắng thừa và chuyển về dạng `snake_case` để thuận tiện cho việc code và tương thích với các hàm xử lý tiếp theo.

2.2.2 `src/data/transformers/`

Package con chứa các class chuyên biệt để xử lý và biến đổi dữ liệu. Mỗi class đảm nhận một nhiệm vụ cụ thể, tuân theo nguyên tắc Single Responsibility.

Class: TypeConverter

Class chuyên xử lý phát hiện và chuyển đổi kiểu dữ liệu.

Methods:

`auto_detect_columns(data)`

Tự động phát hiện và phân loại các cột theo nhóm: *numeric*, *categorical*, *datetime*. Thông tin này được dùng để quyết định chiến lược tiền xử lý phù hợp cho từng loại cột.

`auto_convert_numeric_columns(data, threshold)`

Tự động rà soát các cột kiểu `object` và cố gắng chuyển sang kiểu số. Nếu tỉ lệ giá trị chuyển đổi thành công vượt `threshold` (mặc định 80%), cột đó sẽ được chuyển đổi thành kiểu số.

`convert_to_datetime(data, columns, date_format)`

Chuyển đổi các cột được chỉ định sang kiểu `datetime` theo định dạng ngày tháng cung cấp.

Class: MissingValueHandler

Class chuyên xử lý giá trị thiếu (missing values) trong `DataFrame`.

Methods:

`handle(data, strategies, learned_values, exclude_features, fit)`

Xử lý giá trị khuyết theo chiến lược cấu hình cho từng loại cột (numeric, categorical, datetime). Hỗ trợ chế độ `fit=True` để học tham số từ train set và `fit=False` để áp dụng cho test set, tránh *data leakage*.

Class: OutlierHandler

Class chuyên xử lý giá trị ngoại lai (outliers) trong DataFrame.

Methods:

`clean_negative_values(data)`

Thay thế tất cả giá trị âm trong các cột số bằng giá trị tuyệt đối của chúng. Điều này bám sát quyết định xử lý ở Mục 1, vì các chỉ số dinh dưỡng không nên mang giá trị âm.

`handle(data, method, exclude_features)`

Phát hiện và loại bỏ các hàng chứa giá trị ngoại lai trong cột số sử dụng các phương pháp như IQR, Z-score hoặc Isolation Forest.

Class: FeatureEncoder

Class chuyên mã hóa các cột categorical thành dạng số.

Methods:

`encode(data, strategy, encoders, fit)`

Mã hóa các biến phân loại thành dạng số để mô hình có thể sử dụng. Hỗ trợ Label Encoding và One-Hot Encoding với cơ chế fit/transform riêng biệt cho train/test set.

Class: FeatureScaler

Class chuyên chuẩn hóa (scaling) các cột số.

Methods:

`scale(data, strategy, scaler, exclude_features, fit)`

Chuẩn hóa dữ liệu số về cùng một khoảng (sử dụng `StandardScaler` hoặc `RobustScaler`), khớp với các chiến lược scaling đã trình bày trong Bảng 2.

Class: DataCleaner

Class chuyên thực hiện các thao tác làm sạch dữ liệu cơ bản.

Methods:

`remove_duplicates(data, subset)`

Kiểm tra và loại bỏ các hàng trùng lặp hoàn toàn hoặc trùng trên một tập cột con (subset).

`drop_null_targets(data, target_column)`

Loại bỏ các hàng có giá trị target null, đảm bảo dữ liệu huấn luyện không chứa nhãn bị khuyết.

`drop_features(data, features)`

Loại bỏ hoàn toàn các cột không cần thiết hoặc đã được xác định là gây ra *data leakage*.

2.2.3 src/data/preprocessor.py

Class: DataPreprocessor

Class đóng vai trò *orchestrator* (bộ điều phối), kết hợp `DataIO` và các Transformer class (`TypeConverter`, `MissingValueHandler`, `OutlierHandler`, `FeatureEncoder`, `FeatureScaler`, `DataCleaner`) để tạo thành một pipeline xử lý dữ liệu hoàn chỉnh. Class này lưu giữ trạng thái của dữ liệu trong quá trình biến đổi và hỗ trợ method chaining.

Methods:

`__init__(num_strategy, cat_strategy, dt_strategy, scaling_strategy, outlier_method)`

Khởi tạo đối tượng với các tham số cấu hình cho toàn bộ quy trình: chiến lược xử lý missing values cho từng loại cột (numeric, categorical, datetime), phương pháp scaling và phương pháp xử lý outlier.

`load_data(filepath, auto_convert_numeric, auto_convert_threshold)`

Gọi `DataIO.load_data()` để nạp dữ liệu từ file, thực hiện chuẩn hóa tên cột và tự động chuyển đổi các cột có thể là số nếu `auto_convert_numeric=True`.

`save_data(filepath)`

Lưu trạng thái dữ liệu hiện tại (sau khi xử lý) xuống file CSV, giúp tái sử dụng hoặc chia sẻ cho các bước phân tích khác.

`auto_detect_columns()`

Phân loại các cột (numeric, categorical, datetime) và lưu thông tin này vào thuộc tính nội bộ của class.

`convert_to_datetime(columns, date_format)`

Gọi hàm chuyển đổi ngày tháng và cập nhật lại `DataFrame` nội bộ.

`clean_negative_values()`

Thực hiện làm sạch giá trị âm trên dữ liệu hiện có, đảm bảo tính hợp lý về mặt ngữ nghĩa dinh dưỡng.

`handle_missing_values(data, exclude_features, fit)`

Xử lý giá trị khuyết theo chiến lược đã cấu hình. Hỗ trợ chế độ `fit=True` (học tham số điền trên train set) và `fit=False` (áp dụng cùng tham số đó cho test set) để tránh *data leakage*.

`handle_outliers(data, exclude_features)`

Loại bỏ các hàng chứa giá trị ngoại lai dựa trên phương pháp đã được cấu hình (`outlier_method`).

`encode_categorical(data, strategy, fit)`

Mã hóa biến phân loại và lưu trữ các encoder để áp dụng nhất quán cho dữ liệu mới.

`scale_features(data, exclude_features, fit)`

Chuẩn hóa dữ liệu số, lưu lại `scaler` đã `fit` để dùng cho tập kiểm tra hoặc dữ liệu triển khai thực tế.

`remove_duplicates(subset)`

Loại bỏ các hàng trùng lặp trên dữ liệu hiện tại.

`drop_null_targets(target_column)`

Loại bỏ các hàng có giá trị target null, đảm bảo dữ liệu huấn luyện hợp lệ.

`drop_features(features_to_drop)`

Xóa các cột không còn cần thiết (ví dụ: `calories_from_fat`, `weight_watchers_pnts`, `company`, `item` như phân tích ở mục trước).

`get_processed_data()`

Trả về `DataFrame` kết quả sau khi đã trải qua toàn bộ pipeline tiền xử lý.

2.3 Package: `src.models`

Package này quản lý vòng đời của mô hình học máy: từ huấn luyện, tối ưu siêu tham số cho đến đánh giá và lưu mô hình.

2.3.1 `src/models/evaluator.py`

Class: `ModelEvaluator`

Class tiện ích (utility) cung cấp các phương thức tính để tính toán các chỉ số đánh giá hiệu quả mô hình.

Methods:

`evaluate_model(model, X_test, y_test, model_name)`

Dự đoán trên tập test và tính toán các metrics: MSE, RMSE, MAE, R^2 . Kết quả được trả về dưới dạng `dict` để dễ lưu trữ và phân tích.

`get_feature_importance(model, feature_names, model_name, top_n)`

Trích xuất độ quan trọng của các đặc trưng từ mô hình đã huấn luyện (hỗ trợ cả mô hình tuyến tính với `coef_` và tree-based với `feature_importances_`). Kết quả này liên kết trực tiếp với phân phân tích ở Hình 11.

2.3.2 src/models/io.py

Class: ModelIO

Class chịu trách nhiệm *serialization* (lưu trữ) và *deserialization* (khôi phục) các mô hình và kết quả đánh giá.

Methods:

`load_model(filepath, method)`

Đọc file mô hình từ ổ cứng và khôi phục thành object Python. Hỗ trợ các phương thức: 'joblib' (mặc định) và 'pickle'.

`save_model(model, model_name, filepath, method)`

Lưu object mô hình xuống file để tái sử dụng sau này. Nếu `filepath=None`, tự động tạo tên file với `timestamp` trong thư mục 'outputs/models/'.

`save_results(results, filepath, format)`

Xuất danh sách kết quả đánh giá (metrics) ra file CSV hoặc JSON, giúp liên kết với file Google Sheets trong Mục 4.

2.3.3 src/models/trainer.py

Class: ModelTrainer

Class trung tâm quản lý quy trình huấn luyện, kết nối giữa dữ liệu đã xử lý, các thuật toán mô hình và bước tối ưu siêu tham số. Việc đánh giá và IO được tách ra các module riêng (`ModelEvaluator`, `ModelIO`).

Attributes:

- `best_model_name`: Property trả về tên model có R^2 cao nhất từ `results`.
- `best_model`: Property trả về model object tốt nhất từ `trained_models`.

Methods:

`__init__(random_state, n_jobs)`

Khởi tạo Trainer với `random_state` cố định để đảm bảo tính tái lập (*reproducibility*) và số jobs song song `n_jobs`.

`load_data(data, target_column)`

Nhận dữ liệu đã tiền xử lý và xác định cột mục tiêu `target_column` (trong project này là 'saturated_fat_g').

`split_data(test_size, stratify)`

Chia dữ liệu thành tập Train/Test theo tỉ lệ chỉ định, hỗ trợ `stratify` nếu cần. Giải phóng bộ nhớ data gốc sau khi split.

`set_training_data(train_processed, test_processed, target_col)`

Thiết lập dữ liệu đầu vào (X) và nhãn (y) cho quá trình huấn luyện từ các `DataFrame` đã xử lý. Tách X và y, giải phóng `train_df`, `test_df`.

`initialize_models()`

Khởi tạo danh sách các mô hình sẽ dùng: `ElasticNet`, `RandomForest`, `LightGBM`, `XGBoost`, `DecisionTree`.

`train_models(models_to_train)`

Thực hiện vòng lặp `fit` cho các mô hình được yêu cầu.

`optimize_params(model_name, n_trials, cv, n_jobs)`

Sử dụng Bayesian Optimization (thông qua `Optuna`) để tìm bộ siêu tham số tốt nhất cho một mô hình cụ thể.

`evaluate_models()`

Đánh giá toàn bộ các mô hình đã huấn luyện trên tập Test bằng `ModelEvaluator` và tự động xác định mô hình tốt nhất dựa trên R^2 .

`get_feature_importance(model_name, top_n)`

Lấy danh sách các đặc trưng quan trọng nhất ảnh hưởng đến kết quả dự đoán của mô hình đó. Nếu không chỉ định `model_name`, sử dụng `best_model`.

2.3.4 src/models/optimizer.py

Class: BayesianOptimizer

Class chuyên biệt thực hiện tối ưu hóa siêu tham số sử dụng thư viện `Optuna`. Định nghĩa sẵn không gian tìm kiếm (Search Space) cho từng loại model và sử dụng Bayesian Optimization để tìm kiếm tham số tối ưu.

Methods:

`__init__(X_train, y_train, random_state, cv, n_jobs)`

Khởi tạo Optimizer với dữ liệu huấn luyện, `random_state` cho reproducibility, số fold cross-validation `cv` và số jobs song song `n_jobs`.

`optimize(model_name, n_trials)`

Định nghĩa không gian tìm kiếm (search space) cho từng loại model (`RandomForest`, `LightGBM`, `XGBoost`, `ElasticNet`, `DecisionTree`) và chạy quá trình tối ưu hóa với `direction='maximize'` để tối đa hóa R^2 trên tập validation.

2.4 Package: src.visualization

Package này cung cấp các công cụ trực quan hóa giúp hiểu rõ dữ liệu và kết quả mô hình thông qua các biểu đồ, gắn với các hình đã trình bày ở phần trước.

2.4.1 src/visualization/eda.py

Class: EDA

Class thực hiện *Exploratory Data Analysis* (Phân tích Dữ liệu Khám phá), giúp có cái nhìn tổng quan về dữ liệu trước khi mô hình hóa.

Methods:

`__init__(data, show_plots)`

Khởi tạo với dữ liệu cần phân tích và cờ `show_plots` để quyết định có hiển thị trực tiếp biểu đồ hay chỉ lưu file.

`summary_statistics(save_path)`

Thực hiện phân tích toàn diện: thông tin dataset, thống kê mô tả các cột số, phân tích giá trị khuyết (với barplot), phân tích dòng trùng lặp, tần suất giá trị và độ lệch (skewness) của dữ liệu.

`data_distribution(save_path)`

Vẽ biểu đồ histogram kết hợp đường cong mật độ (KDE) cho tất cả các cột số trên cùng một figure với nhiều subplots, như Hình 2.

`boxplot_analysis(save_path)`

Vẽ boxplot cho các đặc trưng số trên cùng một figure với nhiều subplots nhằm trực quan hóa giá trị ngoại lai, như Hình 3.

`pairplot_analysis(target_column, save_path)`

Vẽ biểu đồ scatter plot so sánh các cột số với cột target, sắp xếp theo lưới 3 cột để dễ quan sát mối quan hệ giữa từng đặc trưng và biến mục tiêu, như Hình 4.

`correlation_analysis(method, save_path)`

Tính toán ma trận tương quan và vẽ Heatmap. Hỗ trợ các phương pháp: 'pearson', 'spearman', 'kendall' hoặc 'all' để vẽ cả 3 loại, tương ứng với Hình 5, 6, 7.

`perform_eda(target_column, corr_method, save_path)`

Phương thức tiện ích chạy toàn bộ quy trình EDA theo trình tự: `summary_statistics()`, `data_distribution()`, `boxplot_analysis()`, `pairplot_analysis()`, `correlation_analysis()`.

2.4.2 src/visualization/model_plots.py

Class: ModelVisualizer

Class chuyên biệt để trực quan hóa hiệu năng của các mô hình sau khi huấn luyện.

Methods:

`__init__(evaluation_results)`

Khởi tạo với kết quả đánh giá thu được từ `ModelTrainer.evaluate_models()` (dictionary chứa 'results' và 'best_model_name').

`plot_model_comparison(save_path)`

Vẽ 4 biểu đồ cột so sánh các chỉ số MSE, RMSE, MAE, R^2 giữa các mô hình trên cùng một figure, highlight model tốt nhất bằng màu khác, tương ứng với Hình 9.

`plot_feature_importance(importance_df, save_path)`

Vẽ biểu đồ thanh ngang thể hiện mức độ đóng góp của từng đặc trưng vào quyết định của mô hình. Tự động sắp xếp theo importance giảm dần, tương ứng với Hình 11.

3 Tổng quan thư viện và phương pháp tối ưu siêu tham số

3.1 Thư viện học máy sử dụng

Trong nghiên cứu này, bốn thư viện chính được sử dụng gồm **Scikit-learn** (**sklearn**), **LightGBM**, **XGBoost** và **Optuna**.

Sklearn là thư viện phổ biến trong Python, cung cấp đầy đủ các thuật toán học máy truyền thống như hồi quy tuyến tính, cây quyết định và các phương pháp đánh giá mô hình.

LightGBM là thư viện chuyên dụng cho các mô hình cây tăng cường (Gradient Boosting), có tốc độ huấn luyện nhanh và hiệu quả cao với tập dữ liệu lớn.

XGBoost là thư viện học máy mạnh mẽ dựa trên cây quyết định, nổi tiếng với khả năng xử lý dữ liệu khuyết và tối ưu hóa hiệu suất tính toán.

Optuna là thư viện tối ưu siêu tham số tự động, hỗ trợ nhiều chiến lược tìm kiếm tiên tiến.

3.2 Các mô hình sử dụng và siêu tham số chính

Model: ElasticNet

Mô hình hồi quy tuyến tính kết hợp cả hai loại regularization L1 (Lasso) và L2 (Ridge), giúp cân bằng giữa feature selection và giảm overfitting.

Siêu tham số chính:

`alpha`

Hằng số nhân với penalty terms, điều khiển độ mạnh của regularization.
Default: 1.0

`l1_ratio`

Tỉ lệ trộn giữa L1 và L2 penalty.

- `l1_ratio = 0`: chỉ có L2 penalty (Ridge)
- `l1_ratio = 1`: chỉ có L1 penalty (Lasso)
- `0 < l1_ratio < 1`: kết hợp cả L1 và L2

Default: 0.5

Model: DecisionTree (DecisionTreeRegressor)

Mô hình cây quyết định đơn lẻ, chia dữ liệu thành các nhánh dựa trên các ngưỡng của features để dự đoán giá trị liên tục.

Siêu tham số chính:

`max_depth`

Độ sâu tối đa của cây.

Default: None (không giới hạn, mở rộng cho đến khi tất cả lá thuần khiết)

`min_samples_split`

Số lượng mẫu tối thiểu để một nút có thể chia.

Default: 2

`min_samples_leaf`

Số mẫu tối thiểu cần có ở mỗi lá (leaf node).

Default: 1

`max_features`

Số lượng features được xem xét khi tìm split tốt nhất.

- None: xem xét tất cả features
- 'sqrt': $\sqrt{n_features}$
- 'log2': $\log_2(n_features)$

Default: None (sử dụng tất cả features)

Model: RandomForest (RandomForestRegressor)

Mô hình Random Forest là một thuật toán ensemble learning dựa trên nhiều cây quyết định, hoạt động song song và kết hợp kết quả thông qua voting để cải thiện độ chính xác và giảm overfitting.

Siêu tham số chính:

`n_estimators`

Số cây trong rừng.

Default: 100

`max_depth`

Độ sâu tối đa mỗi cây.

Default: None (không giới hạn độ sâu)

`min_samples_split`

Số lượng mẫu tối thiểu để một nút có thể chia.

Default: 2

`min_samples_leaf`

Số mẫu tối thiểu ở mỗi lá (leaf).

Default: 1

`max_features`

Số lượng (hoặc tỉ lệ) đặc trưng được xem xét khi tìm split mỗi nút.
Default: 1.0 (sử dụng tất cả features)

Model: LightGBM (LGBMRegressor)

LightGBM là thư viện gradient boosting hiệu suất cao, sử dụng kỹ thuật leaf-wise tree growth và histogram-based learning để tối ưu tốc độ và bộ nhớ.

Siêu tham số chính:

`n_estimators`

Số boosting trees (số vòng boosting).
Default: 100

`learning_rate`

Tốc độ học (shrinkage).
Default: 0.1

`num_leaves`

Số lá tối đa mỗi cây cơ bản (base learner).
Default: 31

`max_depth`

Độ sâu tối đa mỗi cây (nếu ≤ 0 là không giới hạn).
Default: -1

`min_child_samples`

Số mẫu tối thiểu cần ở mỗi lá (leaf).
Default: 20

`subsample`

Tỉ lệ mẫu sử dụng cho mỗi boosting round.
Default: 1.0 (dùng tất cả mẫu)

`colsample_bytree`

Tỉ lệ cột (features) được chọn mỗi cây.
Default: 1.0 (dùng tất cả features)

`reg_alpha`

Regularization L1 (trên weights).
Default: 0.0

`reg_lambda`

Regularization L2 (trên weights).
Default: 0.0

Model: XGBoost (XGBRegressor)

XGBoost là thuật toán gradient boosting mạnh mẽ, tối ưu hóa cả tốc độ và hiệu suất thông qua regularization, parallel processing và tree pruning.

Siêu tham số chính:

`n_estimators`

Số cây / rounds boosting.

Default: 100

`learning_rate` (eta)

Tốc độ học.

Default: 0.3

`max_depth`

Độ sâu tối đa mỗi cây.

Default: 6

`min_child_weight`

Tổng weight (hessian) tối thiểu cần có ở child (leaf).

Default: 1

`subsample`

Tỉ lệ mẫu (rows) sử dụng cho mỗi cây.

Default: 1.0

`colsample_bytree`

Tỉ lệ cột (features) sử dụng mỗi cây.

Default: 1.0

`reg_alpha`

L1 regularization trên weights.

Default: 0.0

`reg_lambda`

L2 regularization trên weights.

Default: 1.0

`gamma` (min_split_loss)

Giá trị giảm loss tối thiểu để phép tách thêm node.

Default: 0.0

3.3 Chỉ số đánh giá mô hình

Hiệu quả của các mô hình được đánh giá thông qua bốn chỉ số chính, mỗi chỉ số cung cấp góc nhìn khác nhau về khả năng dự đoán của mô hình.

Metric: Mean Squared Error (MSE)

Chỉ số đo lường trung bình bình phương của sai số giữa giá trị dự đoán và giá trị thực tế.

Công thức:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Trong đó: y_i là giá trị thực tế, $\hat{y}_i$ là giá trị dự đoán, n là số lượng mẫu.

Đặc điểm và ứng dụng:

- **Phạt nặng sai số lớn:** Do sử dụng bình phương, MSE làm cho các sai số lớn có trọng số cao hơn nhiều so với sai số nhỏ.
- **Nhạy cảm với ngoại lai:** Giá trị ngoại lai (outliers) có thể làm tăng MSE đáng kể, do đó cần xử lý outliers trước khi đánh giá.
- **Đơn vị đo:** MSE có đơn vị là bình phương của đơn vị biến mục tiêu (ở đây là g^2), khó diễn giải trực quan.
- **Khi nào sử dụng:** Phù hợp khi ta muốn ưu tiên giảm thiểu các sai số lớn, đặc biệt quan trọng trong các bài toán mà sai số lớn gây hậu quả nghiêm trọng.

Metric: Root Mean Squared Error (RMSE)

Căn bậc hai của MSE, đưa sai số về cùng đơn vị với biến mục tiêu để dễ diễn giải hơn.

Công thức:

$$RMSE = \sqrt{MSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

Đặc điểm và ứng dụng:

- **Diễn giải trực quan:** RMSE có cùng đơn vị với biến mục tiêu (gram), dễ hiểu hơn MSE.
- **Ý nghĩa thực tế:** Trong project này, RMSE cho biết trung bình mô hình dự đoán sai bao nhiêu gram chất béo bão hòa.
- **Kế thừa đặc tính MSE:** Vẫn phạt nặng sai số lớn và nhạy cảm với outliers.
- **So sánh mô hình:** RMSE thấp hơn nghĩa là mô hình dự đoán chính xác hơn.

Metric: Mean Absolute Error (MAE)

Chỉ số đo lường trung bình giá trị tuyệt đối của sai số, xử lý tất cả các sai số một cách công bằng.

Công thức:

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Đặc điểm và ứng dụng:

- **Xử lý đồng đều:** MAE đối xử công bằng với mọi sai số, không phân biệt lớn hay nhỏ.
- **Ổn định với outliers:** Ít bị ảnh hưởng bởi các giá trị ngoại lai hơn so với MSE/RMSE.
- **Diễn giải đơn giản:** MAE cho biết trung bình mô hình sai lệch bao nhiêu gram so với giá trị thực tế.
- **Khi nào sử dụng:** Phù hợp khi tất cả các loại sai số đều quan trọng như nhau, hoặc khi dữ liệu có nhiều outliers tự nhiên.
- **Ví dụ thực tế:** Nếu $MAE = 2.5g$, nghĩa là trung bình mô hình dự đoán sai lệch 2.5 gram chất béo bão hòa so với thực tế.

Metric: R-squared (R^2) - Hệ số Xác định

Chỉ số đo lường tỷ lệ phương sai của biến mục tiêu được giải thích bởi mô hình.

Công thức:

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}} = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

Trong đó: SS_{res} là tổng bình phương phần dư (residual sum of squares), SS_{tot} là tổng bình phương toàn phần (total sum of squares), $\bar{y}$ là giá trị trung bình của biến mục tiêu.

Đặc điểm và ứng dụng:

- **Thang đo chuẩn hóa:** R^2 nằm trong khoảng $[0, 1]$, với 1 là hoàn hảo và 0 là mô hình không tốt hơn việc dự đoán bằng giá trị trung bình.
- **Diễn giải trực quan:** $R^2 = 0.85$ có nghĩa là 85% biến động của chất béo bão hòa được giải thích bởi các đặc trưng trong mô hình.
- **Đánh giá toàn diện:** R^2 cao (> 0.7) cho thấy mô hình có khả năng dự đoán tốt.
- **Độc lập với đơn vị:** Không phụ thuộc vào đơn vị đo của biến mục tiêu, dễ so sánh giữa các bài toán khác nhau.
- **Chỉ số chính:** Trong project này, R^2 được sử dụng làm chỉ số chính để xác định mô hình tốt nhất và tối ưu hóa siêu tham số.

Tổng kết: Trong nghiên cứu này, chúng ta sử dụng kết hợp cả bốn chỉ số để có cái nhìn

toàn diện về hiệu năng mô hình. R^2 được ưu tiên làm chỉ số chính để so sánh và lựa chọn mô hình, trong khi MSE, RMSE và MAE cung cấp thông tin chi tiết về độ lớn sai số dự đoán.

3.4 Tối ưu siêu tham số bằng phương pháp Bayesian

3.4.1 Cách hoạt động của Bayesian Optimization

Bayesian Optimization không duyệt toàn bộ không gian như *Grid Search* và cũng không lựa chọn ngẫu nhiên như *Random Search*. Thay vào đó, phương pháp này xây dựng một *mô hình xác suất* (surrogate model), chẳng hạn như Gaussian Process hoặc Tree-structured Parzen Estimator (TPE), nhằm xấp xỉ hàm mục tiêu.

1. **Khởi tạo:** Chọn một số bộ siêu tham số ngẫu nhiên và đánh giá hàm mục tiêu tại các điểm này.
2. **Xây dựng mô hình surrogate:** Sử dụng dữ liệu thu thập được để huấn luyện mô hình xác suất mô phỏng hành vi của hàm mục tiêu.
3. **Hàm thu nhận (acquisition function):** Dựa trên mô hình surrogate, hàm này xác định điểm tiếp theo cần thử, cân bằng giữa *khai thác* (exploit) và *khám phá* (exploration).
4. **Cập nhật mô hình:** Sau mỗi lần thử nghiệm, mô hình surrogate được cập nhật và quy trình lặp lại cho đến khi đạt điều kiện dừng.

Nhờ khả năng học từ các lần thử trước, Bayesian Optimization đưa ra các quyết định tìm kiếm có định hướng, thay vì chọn lựa mù quáng.

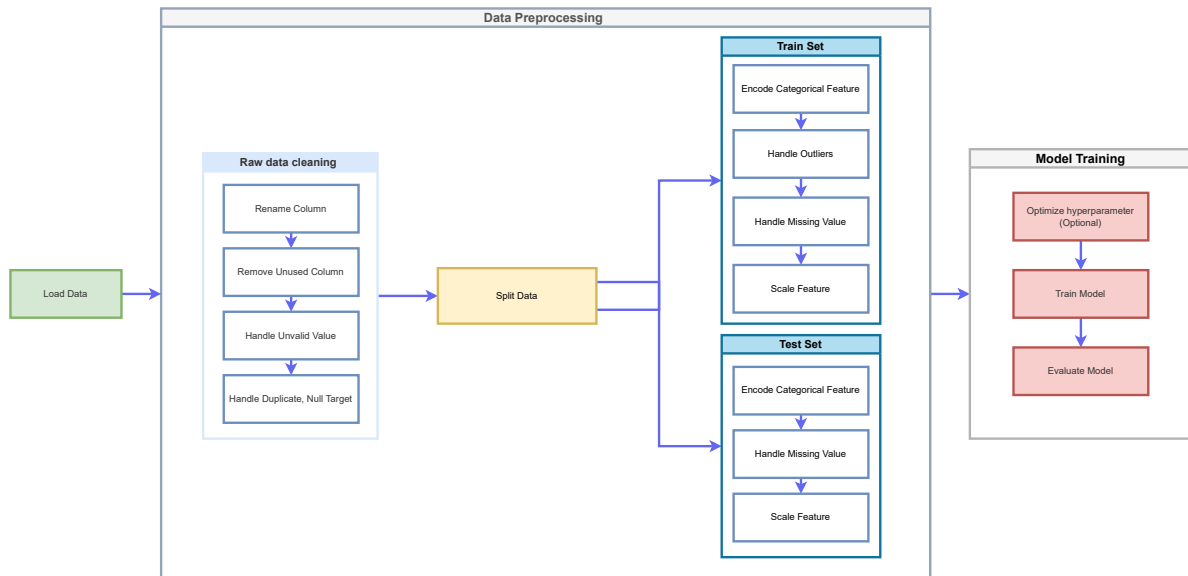
3.4.2 Lý do Bayesian Optimization tốt hơn Grid Search và Random Search

Bayesian Optimization mang lại nhiều ưu điểm vượt trội:

- **Hiệu quả về số lần thử nghiệm:** Giảm đáng kể số lần chạy mô hình cần thiết để đạt kết quả tối ưu, đặc biệt hữu ích khi huấn luyện mô hình tốn nhiều thời gian.
- **Cân bằng khai thác và khám phá:** Hàm thu nhận giúp tập trung vào các vùng có hiệu suất cao đồng thời vẫn khám phá các vùng mới, tránh rơi vào cực tiểu cục bộ.
- **Thích hợp cho hàm mục tiêu phức tạp:** Hữu ích khi hàm mục tiêu không lồi, nhiều hoặc không có biểu thức tường minh.
- **Linh hoạt với nhiều loại siêu tham số:** Có thể xử lý cả tham số liên tục, rời rạc và các cấu trúc tham số có điều kiện, điều mà Grid Search khó thực hiện.

4 Kết quả thực nghiệm

Toàn bộ quá trình thực nghiệm được minh họa trong Hình 8, bao gồm các bước chính: tiền xử lý dữ liệu, huấn luyện mô hình, đánh giá và trực quan hóa kết quả.



Hình 8: Pipeline tổng quan của project

Sau khi thực nghiệm với tất cả các cấu hình tiền xử lý và mô hình (không thực hiện tối ưu siêu tham số), nhóm thu được kết quả như sau³. Cụ thể:

- **Sheet 1:** Giá trị trung bình các metrics cho từng cấu hình tiền xử lý và mô hình.
- **Sheet 2:** Tên mô hình cho ra kết quả tốt nhất tương ứng với từng phương pháp xử lý dữ liệu.
- **Sheet 3:** Toàn bộ kết quả của tất cả mô hình ở các lần chạy.
- **Sheet 4:** Sắp xếp các kết quả ở **Sheet 1** theo R^2 giảm dần.
- **Sheet 5:** Sắp xếp toàn bộ kết quả ở **Sheet 3** tăng dần, để xác định đâu là phương pháp kém hoặc mô hình kém.

num_strategy	scaling_strategy	outlier_method	r2_score
drop	standard	zscore	0.762583344
median	standard	zscore	0.762583344
mean	standard	zscore	0.760899558
mode	standard	isolation_forest	0.759473055
drop	standard	isolation_forest	0.759182476

Bảng 2: Top 5 cấu hình tiền xử lý dữ liệu có kết quả trung bình tốt nhất

Dựa vào **Sheet 5**, ta nhận thấy mô hình Elastic Net luôn cho ra kết quả tệ nhất có thể ảnh hưởng tới các phương pháp được lựa chọn. Ngoài ra việc loại bỏ outlier bằng phương pháp IQR cho ra kết quả không tốt có thể do việc loại bỏ nhiều outlier hơn (so với zscore) dẫn tới mất một số thông tin quan trọng trong dữ liệu.

³<https://docs.google.com/spreadsheets/d/1DugDKIYezPlDGX1fH01LJ1cdBhQisnCArAE03LLE0dM/edit?usp=sharing>

Do đó, ta lựa chọn cấu hình tiền xử lý gồm: loại bỏ giá trị khuyết (**drop**), chuẩn hoá **standard**, và xử lý giá trị ngoại lai bằng **zscore** để tiến hành tối ưu siêu tham số, trực quan các kết quả.

4.1 Chi tiết quá trình tiền xử lý dữ liệu

Quy trình tiền xử lý được thực hiện theo các bước tuần tự sau:

4.1.1 Làm sạch dữ liệu cơ bản

Trước tiên, nhóm loại bỏ các dòng trùng lặp hoàn toàn và các dòng có giá trị target null:

- **Dòng trùng lặp:** 138 dòng
- **Dòng có target null:** 52 dòng
- **Dòng còn lại:** $1147 - 138 - 52 = 957$ dòng

4.1.2 Chia dữ liệu Train/Test

Dữ liệu được chia thành tập huấn luyện và kiểm tra với tỉ lệ 80/20:

- **Tập train:** 765 dòng
- **Tập test:** 192 dòng

Phân tích giá trị khuyết trong tập Train và Test:

- **Tập train:** 26 dòng có giá trị khuyết
- **Tập test:** 9 dòng có giá trị khuyết

4.1.3 Xử lý giá trị ngoại lai

Các giá trị ngoại lai được phát hiện và loại bỏ khỏi tập Train bằng phương pháp Z-score:

- **Tập train:** 81 dòng giá trị ngoại lai được phát hiện và loại bỏ
- **Tập test:** Không loại bỏ giá trị ngoại lai để duy trì tính chân thực của dữ liệu kiểm tra

Sau khi loại bỏ giá trị ngoại lai, tập train không còn dòng nào có giá trị khuyết (các dòng này được xóa cùng lúc).

4.1.4 Xử lý giá trị khuyết

Như đã trình bày ở trên, sau khi loại bỏ giá trị ngoại lai, tập train không còn dòng nào có giá trị khuyết. Do đó, ta chỉ cần xử lý giá trị khuyết trong tập test. Các giá trị khuyết được điền bằng median của từng cột tương ứng trong tập train (sau khi loại bỏ giá trị ngoại lai).

4.1.5 Tóm tắt kết quả tiền xử lý

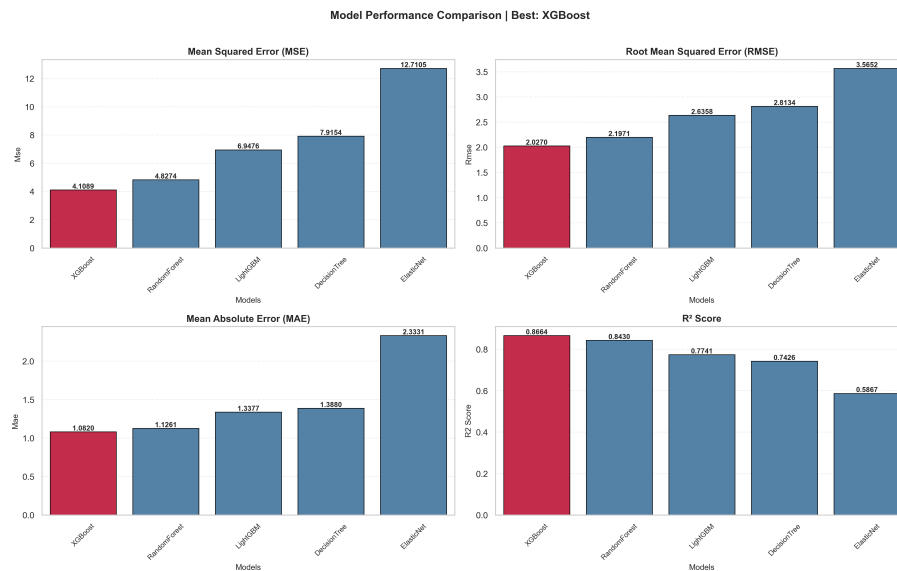
Sau khi hoàn tất toàn bộ quy trình tiền xử lý:

Giai đoạn	Train	Test
Ban đầu	765	192
Sau loại bỏ outliers	684	192
Sau xử lý missing values	684	192

Bảng 3: Số lượng dòng dữ liệu qua các giai đoạn tiền xử lý

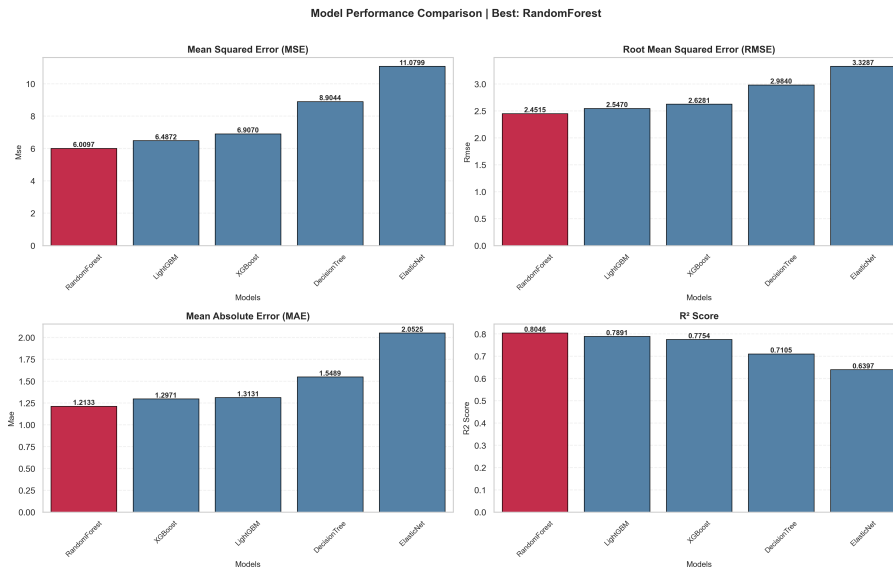
Dữ liệu cuối cùng: 684 dòng train và 192 dòng test, sẵn sàng cho bước huấn luyện mô hình.

4.2 So sánh các mô hình học máy



Hình 9: Kết quả so sánh các mô hình học máy trước khi tối ưu siêu tham số

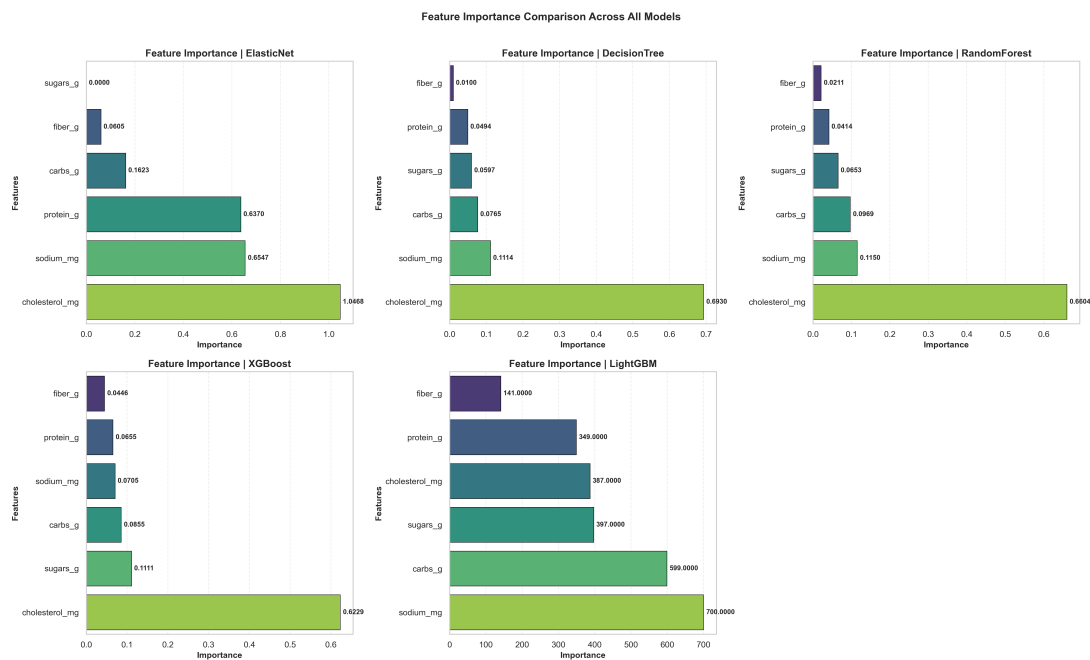
Quan sát Hình 9, **XGBoost Regression** là mô hình có hiệu năng tốt nhất trên cả 4 metrics, với R^2 đạt 0.8664 có thể nói rằng mô hình này dự đoán khá tốt với bộ dữ liệu hiện tại.



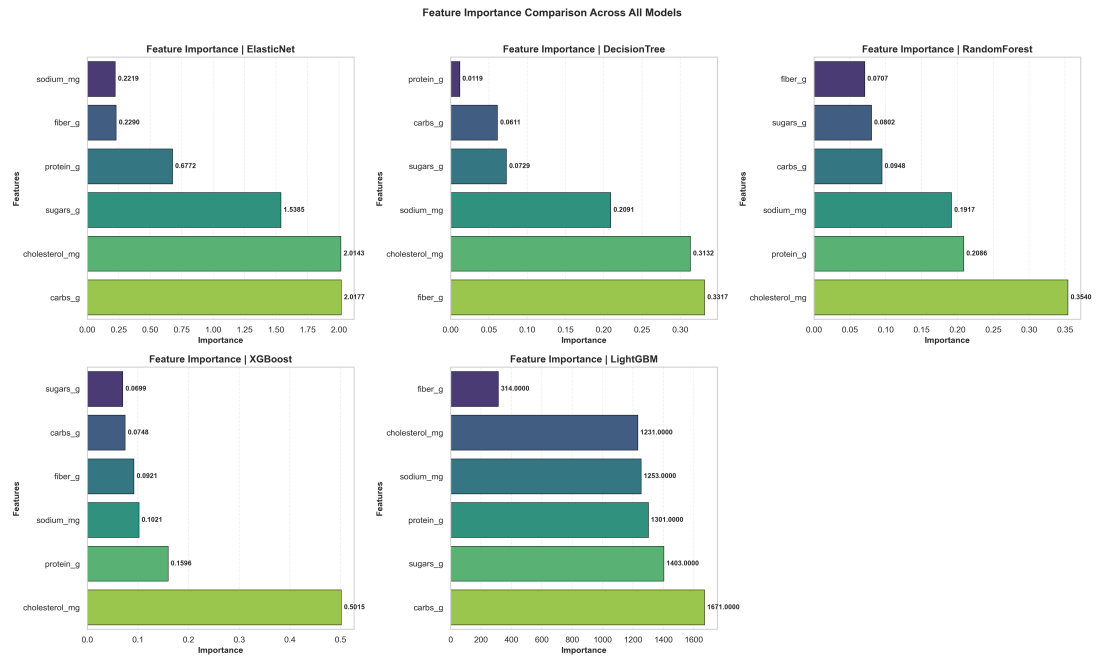
Hình 10: Kết quả so sánh các mô hình học máy sau khi tối ưu siêu tham số

Khi thực hiện tối ưu siêu tham số với hàm mục tiêu là tối đa hoá R^2 , ta thu được kết quả như trong Hình 10. Chỉ có **LightGBM Regression** và **ElasticNet** là cải thiện hiệu năng, trong khi các mô hình còn lại đều giảm. Đáng kể nhất là **XGBoost Regression** giảm từ 0.8664 xuống còn 0.7754. Nguyên nhân có thể do mô hình bị overfitting trong quá trình tối ưu siêu tham số (với 5-fold cross validation), dẫn tới hiệu năng trên tập Test giảm sút.

4.3 Phân tích đặc trưng quan trọng của các mô hình



Hình 11: Các đặc trưng quan trọng theo đề xuất của 5 mô hình trước khi tối ưu



Hình 12: Các đặc trưng quan trọng theo đề xuất của 5 mô hình sau khi tối ưu

Từ Hình 11, có tới 4 mô hình đồng thuận rằng các đặc trưng **cholesterol_mg** là yếu tố quan trọng nhất ảnh hưởng đến hàm lượng **saturated_fat_g**.

Mặc dù **sugars_g** không có độ tương quan cao với mục tiêu, trái lại được mô hình XGBoost đánh giá là quan trọng gấp 3 lần so với **fiber_g** là đặc trưng có tương quan mạnh khi chúng ta phân tích ở Hình 5, Hình 6 và Hình 7. Có thể mô hình XGBoost đã phát hiện ra mối quan hệ giữa **sugars_g** và **saturated_fat_g** do đó nó có kết quả tốt nhất, điều này càng củng cố hơn khi ta phân tích đặc trưng ở Hình 12, **sugars_g** được đánh giá khá thấp hơn dẫn tới kết quả tệ.

5 Bảng công việc

Tên	MSSV	Công việc
Phạm Khang Bình	23280043	- Tổng hợp, tổ chức lại các thuộc tính, phương thức và lớp - Tổ chức mã nguồn phù hợp - Chỉnh sửa docstrings và annotations - Viết pipeline cho toàn bộ chương trình, phần thực nghiệm - Viết báo cáo
Nguyễn Hoàng Anh Duy	23280051	- Xây dựng phần tiền xử lí: DataPreprocessor - Xây dựng phần EDA: Visualization/EDA - Viết pipeline EDA - Bổ sung tham số (argparse) - Chỉnh sửa docstrings và annotations - Hỗ trợ chạy thực nghiệm - Chuẩn bị nội dung thuyết trình
Nguyễn Hải Lâm	23280068	- Xây dựng phần mô hình: ModelTrainer - Xây dựng phần trực quan kết quả: ModelVisualizer - Viết pipeline huấn luyện, tối ưu tham số - Giúp pipeline hỗ trợ truyền config và tham số linh hoạt (argparse) - Chỉnh sửa các phương thức, đảm bảo nguyên tắc vàng - Chuẩn bị nội dung thuyết trình